

THE COVERAGE JOB — four layers, in order, with the trap named at each step

Version 3.40.0 · 2026-08-05 · Authorized by Will: "im gonna turn on feeble for you to handle this, this is a big job." Standard set by Will the same night: completeness, not usage-triage — "WE NEED ALL FUNCTIONS IN THE GAME TO MAKE IT WORK. OTHERWISE ITS PLAYING BLIND MAN."

AMENDED 2026-08-05 after the re-examination Will ordered (see `docs/COVERAGE-PLAN-REVIEW.md` for the full argument). The three material changes: (1) mutation now ships BEFORE the registry — the original stub defense routed stubs into UNREACHED, the one bucket the ratchet deliberately does not guard, so mutation is the registry's tripwire and must exist first; (2) the mutation operator set gains per-PARAM perturbation and a derived-set rebuild hook — tag-level removal cannot see a read-and-ignored param (the WIRE 71 shape), and `medicham2-browser.js` builds its tag-derived sets at module load, so `__setDB` alone would silently no-op; (3) the endgame exploitability re-run at 58 dims is cancelled by the step-probe arithmetic and replaced by a 4–8-dim reparameterization first. Layer numbering below reflects the swap.

This is the execution plan for ending the class of bug this project keeps finding: a fact exists in the data, and the engine consults it partially, wrongly, or not at all — with nothing failing. `docs/TAG-COVERAGE.md` holds the analysis and literature; this file holds the ORDER, the GATES, and the honest failure modes. Read both before touching anything.

The standing gate on all of it

No check is committed until it has been shown failing on a known-bad input. Tonight's split: every instrument that met this bar worked (weather duration: red 4/60 pre-fix; engine release: live edit under an open measurement, snapshot unmoved). Every instrument that skipped it produced garbage (first tag sweep: 132 false DEADs; four successive flinch probes; an inflated exposure number quoted to Will). The difference was never skill on the day — it was whether the failing-arm demonstration was demanded before trusting the green.

Current state (2026-08-05, after the landing pass)

- Census **181 live / 186 probed / 5 missing-with-reasons** (wires 82–89 landed, every probe shown failing against a deliberately broken in-memory engine). Differential 1/150 — the one row is the documented Wood Hammer/Disguise harness-layer mismatch, both engines right.
- `data/interaction-matrix.json` : 1,012 live cases, 999 agree, **13 disagreements remain** (was 68; the 55 were resolved by the wires and the count reproduced on a fresh `--full` run).
- Pre-turn class landed (wire 82). **Shell Trap is** `isNonstandard: 'Past'` — **banned in this format**. Its missing tag is the format door working, not a gap; the derivation matches it in the full dex and the consuming half reads the shape, so a future legalization needs no code edit.
- `data/rulebook-collision.json` (ratcheted ≤ 2): 151 comparable facts, 149 agree, **2 clashes**, both now handled at the consumer (wire 89 reads the FORMAT's secondary chance and counts drift). 27 tag-only / 166 fx-only facts could not be compared — that blind mass is Layer 1's real target.
- `data/tag-consumption.json` (ratcheted): 67 LIVE, 18 STAGED, 30 UNREACHED, **61 DEAD** (the 26 ability/item orphans + 35 move tags duplicated by the second rulebook).

- MEASURE's MAG four-channel refit is FINISHED and verified (`fitEnvironment` stamped, 231,722 decisions probed, 99.67% channel reach); the JOINT refit is the pending half. SEARCH's step-rule fix is proven on a planted optimum — and the proof cancels the 58-dim re-run (see the endgame). `data/exploitability.json` is void and must stay void until the reparameterized search runs.

Layer 0 — the queue that is already concrete (no design, just work)

The matrix disagreements (13 remain of the original 68) and the 26 orphan ability/item tags. Each has a reproducible case or a named carrier. Nothing below starts until the 13 are resolved or reclassified, because the matrix is the instrument that will verify everything else.

Layer 1 — ONE GENERATOR for the two rulebooks

Two derivations of the same upstream, one figure per paragraph so each sits beside its own artifact.

`data/tags.json` carries 500 moves.

`CHOMP/data/move-effects.json` carries 954 moves and is THE one the engine reads for `flinch/status/secondary/recoil`.

(Both counts sat on ONE line until 2026-08-09. `tests/test-docs-current.js` rule 3b pairs a figure with an artifact cited on the same line, so the move-effects count was being checked against the tag artifact, which happened to contain that string as a usage count until it was regenerated. The attribution was always wrong and the coincidence was hiding it. One figure, one line, one artifact.) Decision made: **neither wins — one derivation, two emitted shapes**, so disagreement becomes impossible. CHOMP keeps its file format; ABRA keeps its tags; one script writes both.

- **The deflation measurement RAN (2026-08-04): 2 clashes in 151 comparable facts.** Iron Head was the whole story in one row — tags carried the format's 20% flinch, move-effects the generic 30%, and the engine read the wrong one until wire 89. This layer is therefore INSURANCE, built but no longer cited as urgent. The real remaining exposure is the **27 tag-only + 166 fx-only facts the comparison could not reach** — the unified generator exists to close that blind mass, and the extended comparison over it is part of this layer's deliverable.
- Gate: the merged generator reproduces BOTH current files byte-identically (modulo agreed fixes from the collision report) before it replaces either. This is a golden-master gate; treat it as one (any intended divergence is named in the CHANGELOG, never absorbed).
- **The 35 duplicated move tags get a generated fate.** The generator emits a `carried-by-other-output` table naming every fact it deliberately routes to only one output. The registry (Layer 3) honors that table, so the 35 do not become 35 load failures and the declaration cannot rot the way a hand list rots.

Layer 2 — MUTATION: prove each handler MATTERS (*moved ahead of the registry, 2026-08-05*)

Mutation moved up and the registry moved down, for a reason the original plan got wrong: it claimed a stub handler "shows up in `asked()` / `hits()` as a live tag with zero reads, and `test-tag-consumed.js` ratchets it." It does not. A registered handler that never fires is *named in source with ASKED = 0* — that is UNREACHED, and UNREACHED is deliberately never ratcheted because it measures the sweep, not the engine. A stub that fires and ignores its payload reads LIVE. Stubs land precisely in the buckets the ratchet does not guard.

Mutation is the only instrument that catches a stub, so it exists before the registry creates the pressure to write one. WIRE 71 is the standing example: handlers on all four routes, three wrote the wrong constant, every test green.

- Method (spec in TAG-COVERAGE.md §2): `__setDB()` injection point in `engine/tags.js` (do NOT clear the require cache — it drops the counters), **plus a derived-set rebuild hook**: `medicham2-browser.js` builds `SPREAD`, `HITS_ALLY`, the terrain table and the priority-block map at module load, so an injected DB alone silently no-ops for every set-building tag and scores it read-and-ignored — a false DEAD, the dangerous direction. The hook re-runs the set builders; the gate below proves it fires.
- **Operators are per-tag AND per-param.** Remove tag *T*; separately, perturb each param (flip booleans, write a sentinel numeric). Tag-level removal cannot see a read-and-ignored param — removing `spreadAll` moves the digest via the set while `hitsAlly` stays ignored — and the param level is where WIRE 71 actually lived. Both operator families come free from the tag list; cost stays $O(\text{operators} \times \text{battery})$, fine at seconds per run.
- **A small seed battery per mutant, not one seed.** One seed can miss a probabilistic effect (false DEAD); and a removed tag shifts PRNG consumption so state diverges for unrelated reasons (false LIVE — benign, but it inflates confidence and the artifact says so). State identical → check whether the REFERENCE engine's arms differ (the interaction-matrix trick — the equivalent-mutant defense) before calling it read-and-ignored.
- Gate: the harness is validated by planting one known read-and-ignore **at the param level** (`spreadAll`'s `hitsAlly` if still unfixed) AND one set-building tag, and confirming both are caught. The original gate named only `hitsAlly` while specifying an operator that could not catch it — the gate would have failed its own harness.

Layer 3 — THE REGISTRY: a fact with no consumer fails at LOAD (after mutation)

The exhaustiveness check the architecture always needed. Not a test that reports — a load that refuses. A red report gets named a "known failure" and lived with (this repo did exactly that for two days across ~40 commits); an engine that will not start cannot be lived with.

- **Scope: the unified fact model, not the tags output alone.** After Layer 1 one generator emits two shapes, and the engine reads `move-effects` for flinch/status/secondary — a field there with no consumer must fail the same way a tag with no handler does. Exhaustiveness over tags only would guard the copy the engine reads less.
- **The stub trap, now with a real tripwire:** Layer 2's mutation harness is standing before the registry turns on, so a handler that exists but changes nothing is caught by measurement, not hoped against by a ratchet that deliberately excludes it.
- **Sequencing trap:** the registry cannot ship while the 26 orphans exist (Layer 0 clears them) — nor while the 35 duplicated move tags have no fate; the generated `carried-by-other-output` table (Layer 1) is their declaration, so day one is 0 failures without one hand-written stub.
- Hardcoded names (intimidate, prankster) get rewritten to read the artifact as part of this layer — with the registry on, a name-only implementation is structurally impossible to mistake for coverage, because the tag still has no handler.
- **Ops story, because fail-at-load is outward-facing:** a tags regeneration that derives a new tag must brick the pre-deploy smoke in `tests/run-all.js`, not the live bot at 2am — and when the bot or the site page does refuse to load, the refusal names the unhandled fact. A refusal nobody can see is the old failure mode wearing the new fix.
- Gate: registry on, engine loads, `test-tag-consumed.js` DEAD count is 0, and deleting any single handler makes the engine refuse to start (that is the known-bad-input demonstration).

Layer 4 — what none of the above catches, and what already covers it

A tag derived WRONG from upstream is faithfully propagated by one generator, happily registered, and behaviourally consistent under mutation. Only the differential against the official engine sees it: `tests/test-engine-diff.js` (1/400), `tests/test-game-diff.js` (multi-turn), and the interaction matrix's four-arm design. These stay, run in CI order after the layers land, and the two unexplained Mummy/Wandering-Spirit divergences in the matrix are the standing example of why.

What "done" means

1. Matrix disagreements: 0 remaining (each fixed, or reclassified with the reference engine's evidence). 13 open as of 2026-08-05.
2. Two rulebooks: one generator, collision report published (done, ratcheted ≤ 2), both outputs reproduced, the 193 previously-uncomparable facts brought into the comparison.
3. Mutation pass complete — per-tag and per-param — with every read-and-ignore either fixed or declared with its reason, and the harness validated on both planted cases.
4. Registry on over the unified fact model; DEAD = 0; delete-a-handler refuses to load; the pre-deploy smoke exists.
5. Matrix re-run at `--full`; agreement $\geq$ current 999/1,012 with the denominator not shrunk by reclassification games.
6. **The endgame, rewritten 2026-08-05 because the step probe settled it with arithmetic:** the 58-dim re-run is cancelled — the probe's arithmetic is that one accepted step moves far less than the resolution of the comparison, and that closing a quarter of the planted distance costs a game budget nobody will spend. The step size, the resolution and the game count are **QUARANTINED — withheld, not annotated** (`data/exploit-step-probe.json` , stamped to release d3d04b669e18, is downstream of MEDICHAM); they become quotable again when the gate opens AND this is re-run: `node engine/exploit_step_probe.js` . Instead: reparameterize MAG's policy to a 4–8-number family (the largest searchable at the real budget), cut a fresh release AFTER the layers above land, and run the exploitability search in that space. That number — the one Will has been owed since the retraction — is still the point of the entire job; only the route changed. Until it exists, ABRA has no exploitability number and `data/exploitability.json` stays void.

Division etiquette (unchanged, learned expensively)

Writers may run beside writers. A measurement opens `engine_release` and reads the snapshot — never the live tree. Cut a fresh release after the engine work lands and before any measurement. No agent runs git. No agent deletes a file. A red test is fixed in the session that sees it red or reported by name — never filed.